



भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

ID1063: Introduction to Programming (Lab-6)

Srijith P.K, **Saurabh Kumar**

Dept of CSE

IIT Hyderabad

Lab-6 Questions

Three Questions

Don't forgot to enable
ITP-Monitor Plugin

Q1: Magical Power Number

Saheli loves to experiment with numbers. Recently, she discovered an interesting property: A number is called a Magic Power Number if the sum of its digits raised to the power of the total number of digits equals the number itself.

For example:

$$153 = 1^3 + 5^3 + 3^3 = 153 \Rightarrow \text{Magic Power Number}$$

$$9474 = 9^4 + 4^4 + 7^4 + 4^4 = 9474 \Rightarrow \text{Magic Power Number}$$

Saheli now wants to check how many such numbers exist within a given range.

The rules are as follows:

- ❑ The program must read two integers, start and end, representing the inclusive range.
- ❑ For each number in the range, check whether it is a Magic Power Number.
- ❑ Print each Magic Power Number on a separate line.
- ❑ Finally, print the total count of Magic Power Numbers in the specified format.

Your task is to help Saheli by writing a C program that follows the above rules.

Input Format: The input will be in a single line with two integers separated by a space in the following format:
start end

Q1: Magical Power Number

Output Format: Print each Magic Power Number found in the range on a separate line. After listing them, print one final line showing the total count in the format:

TOTAL: k

where k is the number of Magic Power Numbers found. There will not be any extra spaces or newline characters after the final output.

Note:

1. Must use a function to perform the Magical Power Number.
2. No recursion allowed.
3. Start and End should be strictly given in increasing order.
4. Use positive int for the operation.
5. Implement the isMagicalPower() function.
6. Don't delete the template code provided.

Example 1:

Input

1 200

Output

1

2

3

4

5

6

7

8

9

153

TOTAL: 10

Q2: Musical Harmony Checker

A music academy is organizing a group performance competition. Each contestant performs in multiple rounds, and the judges assign scores for each round. The scores are stored in a square matrix where:

- ❑ Each row represents a contestant.
- ❑ Each number in the row represents the score of that contestant in a particular round.

The organizers want to check if the performance is in perfect harmony:

- ❑ First, calculate and print the total score of each contestant (row sum).
- ❑ If all contestants have the same total score, print Harmonic.
- ❑ Otherwise, print Discord.

Input Format: The first line contains an integer n , the size of the square matrix ($n \times n$). The next n lines contain n integers each, representing the scores of contestants.

Output Format: Print the total score of each contestant (row sum) in separate lines. On the final line, print either Harmonic or Discord. There will not be a newline character at the end of the output.

Constraints:

1. Use functions to structure your program.
2. The matrix should be Square.

Q2: Musical Harmony Checker

Example 1:

Input

3

10 20 30

15 25 20

20 15 25

Output

60

60

60

Harmonic

Q3: The Easy Maze

Harsh is trapped in a $N \times N$ maze. Each cell contains a positive integer representing the move intensity, i.e. the number of blocks Harsh can move. Harsh can only move to the right (like reading a book: wrap to the next line if current gets over). Harsh starts at (0,0) and must reach/cross the bottom-right corner ($N-1, N-1$).

Calculate the number of moves required to reach/cross the goal [i.e. the ($N-1, N-1$)th block.]

Input Format: (scan a 2-D array, i.e. the maze). The first line contains an integer n , the size of the square matrix ($n \times n$). The next n lines contain n integers each, separated by space, representing the move intensity.

Output Format: print number of moves required.

Example:

Input

```
3
1 2 3
3 2 1
1 2 3
```

Output

```
5
```

Q3: The Easy Maze

Explanation:

- ❑ Starting at (0,0) [move intensity = 1] => move one block right to (0,1) = move 1
- ❑ Now from (0,1) [move intensity = 2] => move two blocks right, i.e. first to (0,2), then to (1,0) [i.e. next line] = move 2
- ❑ Now from (1,0) [move intensity = 3] => move three blocks right, i.e. to (1,1), then to (1,2), then to (2,0) = move 3
- ❑ Now from (2,0) [move intensity = 1] => move one block right, i.e. to (2,1) = move 4
- ❑ Now from (2,1) [move intensity = 2] => move two block right, i.e. to (2,2) | reached bottom right corner = move 5

Since it took 5 moves to reach goal, print 5.

Note: In explanation, (x, y) depicts co-ordinates and not necessarily 2-D array indexes.